

National University of Computer & Emerging Sciences (FAST–NUCES), Islamabad

Department of Artificial Intelligence & Data Science

Assignment 1: IncidentZero

Building a Bounded Autonomous SRE Incident Commander from Scratch

Assignment mode	Individual
Marks	100
Required model access	Groq API
Default final-run model	openai/gpt-oss-20b unless changed by instructor announcement
Agent frameworks	Strictly prohibited
AI coding assistants	Permitted, but declared and individually defended in viva

This is not a “build a chatbot” assignment. You are engineering the runtime of an autonomous decision-making system that operates under uncertainty, changing world state, safety rules, human approval, rate limits, and incomplete evidence. A solution that works only on the visible scenario is not a successful solution.

Contents

1	Why This Assignment Exists	3
1.1	Why the SRE/incident-response domain?	3
2	The Real-World Scenario	3
3	What You Are Building	4
3.1	Required separation of responsibilities	4
4	Provided Starter Repository	4
4.1	Protected simulator boundary	5
5	Individualized and Hidden Scenarios	5
5.1	Do not optimize for a fixed root cause	5
6	Local Tool Surface	6
6.1	Observation tools	6
6.2	Action tools	6
7	Mandatory Agent Requirements	6
7.1	R1: Explicit Python state	6
7.2	R2: Explicit initial plan	7
7.3	R3: Evidence before consequential action	7
7.4	R4: Dynamic re-planning	7
7.5	R5: Distinguish retry from re-plan	8
7.6	R6: Optimistic concurrency using <code>world_version</code>	8
7.7	R7: Human approval cannot be generated by the LLM	8
7.8	R8: Tool validation and execution boundary	8
7.9	R9: Loop detection	9
7.10	R10: Verifiable stopping conditions	9

7.11 R11: Groq request handling	9
7.12 R12: Hard internal resource budget	10
7.13 R13: Traceability	10
7.14 R14: Offline testability	10
8 Framework and Library Policy	10
8.1 Strictly prohibited	11
8.2 Allowed	11
8.3 Groq features that are out of scope	11
9 AI Coding Assistants and Academic Integrity	12
9.1 Why copy-paste generation is insufficient	12
10 Implementation Tasks	12
10.1 Task A – Re-planning policy	12
10.2 Task B – Loop guard	12
10.3 Task C – Model retry policy	12
10.4 Task D – Approval integration	13
10.5 Task E – Plan revision	13
10.6 Task F – Controller hardening	13
10.7 Task G – Tests	13
11 Expected Behavioral Examples	13
11.1 Example 1: deployment regression	13
11.2 Example 2: stale world after planning	13
11.3 Example 3: approval denial	13
11.4 Example 4: successful action but failed recovery	13
11.5 Example 5: impossible incident	14
12 Testing and Evaluation Protocol	14
12.1 Public tests	14
12.2 Your three required public evaluation runs	14
12.3 Hidden evaluation philosophy	14
13 Submission Requirements	14
14 Marking Rubric – 100 Marks	15
14.1 Performance is not code volume	16
15 Viva / Live Verification	16
16 Non-Compliance and Safety Conditions	16
17 Frequently Asked Questions	16
18 Recommended One-Week Work Plan	20
19 Final Pre-Submission Checklist	20
20 References and Current Context	21

1 Why This Assignment Exists

In the class notebook, you implemented a small agent in which the LLM proposed an action, Python executed a local tool, the tool result was appended to the conversation, and the model continued until it finished. That example intentionally kept the environment small and visible. The assignment now asks you to retain that transparent architecture while solving a problem that is closer to the type of agentic automation being built in 2026.

You have studied, through Week 4, the foundations of agents, LLM interaction, structured prompting/output, reasoning-and-acting patterns, reflection and failure-aware retries, goal decomposition, plan-and-execute patterns, dynamic re-planning, stopping conditions, failure recovery, and human approval gates. The assignment is designed around exactly those ideas. Tool definitions are supplied in the starter repository, so you are *not* being assessed on future course material such as RAG, long-term memory, LangGraph, MCP, or multi-agent systems.

The difficulty comes from **agent engineering**: what the controller does when the model is wrong, the world changes, a tool fails, approval is denied, the request budget is nearly exhausted, or the apparent solution does not actually restore the system.

1.1 Why the SRE/incident-response domain?

Modern production systems consist of many dependent services, deployments, databases, caches, traffic shifts, telemetry sources, and operational policies. During a production incident, an engineer must gather evidence, form and revise hypotheses, choose a bounded remediation, verify whether it worked, and know when to escalate. This is a strong fit for the concepts taught so far because success cannot be reduced to a single prompt or a fixed sequence of functions.

The domain is also deliberately current. In 2026, major platforms and cloud providers are publicly describing agentic systems for SRE, incident investigation, autonomous operations, bounded remediation, and human-governed action. See the references in Section 20. The assignment uses a fully local simulator so that the pedagogical task is reproducible and safe.

2 The Real-World Scenario

You have joined the reliability engineering team of a fictional cloud commerce platform. The customer-facing checkout path is composed of several microservices and infrastructure components:

Component	Role
<code>edge-gateway</code>	Entry point for customer requests.
<code>auth-service</code>	Authentication and session validation.
<code>catalog-service</code>	Product catalogue lookup.
<code>cart-service</code>	Shopping-cart state.
<code>checkout-service</code>	Coordinates checkout.
<code>payment-service</code>	Performs payment authorization.
<code>inventory-service</code>	Confirms and reserves stock.
<code>order-service</code>	Creates orders.
<code>redis-cache</code>	Shared cache used by customer-facing services.
<code>order-db</code>	Primary order database.

At runtime, your agent receives an incident such as elevated checkout errors, timeout spikes, inconsistent cart state, a recent deployment, resource saturation, database degradation, or an external dependency failure. The incident ticket is **not guaranteed to identify the correct root cause**. It may contain an outdated or misleading first-responder hypothesis.

Your agent is responsible for the following high-level goal:

Investigate the active production incident, construct an evidence-grounded plan, execute only safe and justified remediation actions, adapt if the environment changes, verify objective recovery, and either close the incident with evidence or escalate safely.

3 What You Are Building

You will evolve the supplied baseline into a framework-free autonomous agent with the following logical cycle:

Observe → Plan / Hypothesize → Choose Action → Validate → Approve if Required → Execute → Verify → Re-plan or Stop

The LLM may propose decisions. It is never the authority that executes them. Python remains responsible for validation, routing, budgets, authorization, state updates, retries, and terminal decisions.

3.1 Required separation of responsibilities

Layer	Responsibility
LLM	Interpret evidence, propose hypotheses, generate/revise a plan, select a local tool, and explain a final result.
Python controller	Decide whether the proposed operation is syntactically valid, allowed, budget-feasible, stale, repetitive, approval-gated, retryable, or terminal.
Local tools	Return observations or mutate the simulated environment.
Simulator	Authoritative world state. It determines what actually happened.
Approval gateway	Authoritative human decision for high/critical actions. The LLM cannot self-approve.
Trace recorder	Durable evidence of the agent trajectory, failures, retries, approvals, replans, and outcome.

4 Provided Starter Repository

The starter repository is intentionally substantial. You are not expected to spend the week writing boilerplate. The environment, domain model, tool schemas, Groq adapter, budget helper, trace helper, risk configuration, and public tests are supplied. The baseline agent loop is deliberately weak and must be engineered into a robust runtime.

The repository contains approximately the following structure:

```
incidentzero_starter/
|-- README.md
|-- requirements.txt
|-- pyproject.toml
|-- .env.example
|-- configs/
|   |-- limits.json
|   |-- risk_policy.json
|   '-- services.json
|-- incidentzero/
|   |-- agent/
|   |   |-- controller.py
|   |   |-- planner.py
|   |   |-- policies.py
|   |   |-- recovery.py
|   |   |-- prompts.py
|   |   '-- state.py
```

```
| |-- approval/gateway.py
| |-- domain/models.py
| |-- environment/
| | |-- engine.py
| | |-- scenario_factory.py
| | |-- topology.py
| | |-- models.py
| |-- model/
| | |-- base.py
| | |-- groq_client.py
| | |-- scripted.py
| | |-- errors.py
| |-- telemetry/
| | |-- budget.py
| | |-- trace.py
| |-- tools/
| | |-- definitions.py
| | |-- registry.py
| |-- cli.py
|-- tests/public/
-- scripts/
-- docs/
-- traces/
-- artifacts/
```

4.1 Protected simulator boundary

You may read the simulator code to understand the public contract. However, your runtime must interact with the world through the supplied tool registry. It is forbidden to read private/oracle state such as `_scenario_spec`, `_services`, `_oracle_snapshot()`, or to decode the student seed into a root cause. Hidden grading uses fresh simulator instances and may change private implementation details.

Important: Changing the simulator so that your agent passes a visible case is not solving the assignment. The grader uses clean simulator code and hidden scenario seeds.

5 Individualized and Hidden Scenarios

Every student runs deterministic public scenarios derived from the submitted roll/registration number. The starter uses a cryptographic hash to derive a seed, which changes incident family, initial evidence, transient failures, and environmental events. This provides reproducibility while reducing the value of copied hard-coded solutions.

Use:

```
python scripts/generate_student_scenario.py \
  --student-id 22I-1234 \
  --scenario public-a
```

You must evaluate at least `public-a`, `public-b`, and `public-c`. The grader will additionally use hidden scenario IDs and different seeds. Hidden tests may alter timing, approvals, model replies, budgets, and failure injection.

5.1 Do not optimize for a fixed root cause

A correct controller should work when the incident is caused by a deployment regression, memory leak, service-capacity saturation, degraded database primary, cache corruption, or a condition that cannot be safely corrected by the available tools. You are not told which family will occur in a hidden run.

6 Local Tool Surface

The starter defines the local action space. You may improve descriptions and validation if needed, but you may not introduce external operational APIs.

6.1 Observation tools

Tool	Purpose
<code>get_incident()</code>	Read the incident ticket. Treat suspected causes as unverified.
<code>get_service_health(service)</code>	Obtain current health, replicas, and deployed version.
<code>get_metrics(service)</code>	Obtain error rate, latency, CPU, memory, and component-specific metrics.
<code>get_logs(service, limit)</code>	Obtain recent simulated log evidence.
<code>get_deployments(service)</code>	Compare current and known-good previous release.
<code>get_dependencies(service)</code>	Inspect direct upstream/downstream dependencies.
<code>get_runbook(topic)</code>	Read a short local operational runbook. This is static local knowledge, not RAG.
<code>verify_recovery()</code>	Test objective incident-recovery criteria.

6.2 Action tools

Tool	Purpose	Risk
<code>restart_service</code>	Restart one service.	Medium
<code>scale_service</code>	Change service replica count.	Medium
<code>clear_cache</code>	Invalidate simulated Redis cache.	Medium
<code>rollback_deployment</code>	Roll back to the known previous release.	High
<code>failover_database</code>	Move order DB to a healthy replica.	Critical
<code>shift_traffic</code>	Shift traffic to a secondary service pool.	High
<code>close_incident</code>	Close only after objective recovery evidence.	Medium
<code>escalate_incident</code>	Stop autonomous remediation and escalate with evidence.	Low

Risk levels are loaded from `configs/risk_policy.json`. Your controller must not hard-code a list such as “if rollback then approve.” The policy may change during the viva or hidden evaluation.

7 Mandatory Agent Requirements

This section is the core specification. Read it as a contract.

7.1 R1: Explicit Python state

Your agent must maintain state that exists independently of raw chat messages. At minimum it must track:

- current explicit plan and plan revision number;
- evidence IDs gathered so far;

- latest observed `world_version`;
- recent tool outcomes;
- LLM-call and tool-call budgets;
- repeated action fingerprints or equivalent loop-detection state;
- terminal status: running, resolved, escalated, aborted, budget exhausted, or failed.

A long `messages` list by itself does not satisfy this requirement.

7.2 R2: Explicit initial plan

Before taking a remediation action, the agent must create an explicit plan containing at least:

- a working incident hypothesis;
- two or more investigation/remediation subgoals;
- a success signal for each major step;
- a final verification step;
- enough structure that the controller can record a revision when evidence changes.

The first hypothesis may be wrong. That is acceptable. Failing to revise it when evidence contradicts it is not.

7.3 R3: Evidence before consequential action

A high or critical action must be supported by relevant observed evidence. A model sentence such as “rollback is probably best” is not evidence. Evidence means simulator results such as deployment timing, logs, resource metrics, dependency status, or verification output.

Your trace should make it possible for a TA to answer: *What did the agent know when it chose this action?*

7.4 R4: Dynamic re-planning

Your controller must be able to revise the plan during the run. Re-planning must occur when warranted, including cases such as:

- a stale-world precondition indicates the world changed after observation;
- a high-risk action is denied by the human approver;
- new evidence contradicts the active hypothesis;
- the chosen remediation succeeds technically but `verify_recovery` remains false;
- a non-retryable action fails;
- the remaining call budget is too small to continue the current plan safely.

A re-plan is not “call the LLM again with the same prompt.” It must preserve relevant completed evidence while changing the active hypothesis, remaining steps, or terminal strategy.

7.5 R5: Distinguish retry from re-plan

The assignment intentionally includes multiple kinds of failure. Your solution must not treat all failures as “retry.”

Condition	Expected class of response
Groq/network transient error or synthetic 429	Bounded retry with backoff; consume budget correctly; do not retry forever.
Transient telemetry-tool time-out	Bounded retry or obtain equivalent evidence by another observation path.
Malformed/invalid model tool arguments	Reject before execution; return a corrective observation to the model.
<code>stale_precondition</code>	Re-observe relevant state, then re-confirm or revise the plan. Do not blindly substitute a newer version and fire the old action.
Human approval denied	Do not execute the action. Re-plan or escalate.
Action returns <code>ok</code> but verification fails	Re-plan; success of an API call is not success of the incident.
Permanent model/configuration error	Fail/abort safely rather than infinite retry.
Budget nearly exhausted	Prioritize essential verification or escalate safely before the hard limit.

7.6 R6: Optimistic concurrency using `world_version`

Every observation contains a `world_version`. Consequential actions require an `expected_world_version`. The simulator can change after you observe it and before you act. If the version is stale, the action is rejected.

This requirement models a real distributed-systems problem: decisions can be valid when planned and invalid a moment later. Your agent must use the most recent observed version and must treat staleness as new information.

7.7 R7: Human approval cannot be generated by the LLM

All tools marked High or Critical in the risk policy require a call to the supplied Python approval gateway before execution. The model may explain why it wants the action. It cannot create an approval token or claim “the human approved.”

If the approval gateway returns false:

1. the protected action must not execute;
2. an `approval_denied` observation or equivalent must be recorded;
3. the model/controller must reconsider the plan or escalate.

7.8 R8: Tool validation and execution boundary

Before execution, every tool request must be checked for:

- known tool name;
- JSON/dictionary shape;
- required arguments;
- argument types and permitted values;

- risk/approval requirement;
- remaining tool budget;
- repeated-action/loop policy;
- any relevant state precondition.

An invented tool or extra parameter must never reach the simulator.

7.9 R9: Loop detection

The system must detect unproductive repetition. At minimum, repeated identical action+arguments beyond the configured limit must trigger a different control decision. A strong solution also notices semantically equivalent loops, repeated failed hypotheses, or observation thrashing.

The starter public test expects the exact same action to be flagged after the configured repeat threshold.

7.10 R10: Verifiable stopping conditions

The following are **not** proof of resolution:

- the LLM says “resolved”;
- a remediation action returned **status=ok**;
- latency improved in one component;
- a previous plan predicted that the action should work.

A normal successful run must satisfy:

1. execute **verify_recovery**;
2. obtain **criteria_met=true**;
3. retain the returned evidence ID;
4. execute **close_incident** using the latest valid world version and including the latest verification evidence;
5. receive a successful close result.

If autonomous recovery is impossible or unsafe, **escalate_incident** can be the correct terminal action.

7.11 R11: Groq request handling

Groq is the only external runtime dependency. Your solution must:

- read the API key from environment configuration; never hard-code it;
- use local function/tool calling only;
- handle retryable provider errors including 429 without infinite retries;
- not assume a single provider response is always well formed;
- keep the model name configurable;
- record available request/token usage in the trace when possible.

Provider quotas can differ by account and can change. Therefore, grading uses the assignment’s own deterministic budget in addition to whatever limits Groq applies.

7.12 R12: Hard internal resource budget

Each independent agent run must respect the following default limits from `configs/limits.json`:

Resource	Maximum
LLM requests, including planning/re-planning	14
Local environment tool calls	28
Consecutive retries of one transient model operation	3 attempts total
Identical action repetition before loop intervention	2 repeats allowed; the next identical repeat must be blocked or redirected
Target maximum wall-clock runtime	120 seconds

You may use fewer calls. You may not silently raise the configured limits to make your solution pass.

7.13 R13: Traceability

Each live run must produce a JSONL trace in `traces/`. At minimum, record:

- incident bootstrap evidence;
- plan creation and every plan revision;
- model request outcome and tool proposal;
- validation failures;
- approval request and approval result;
- tool result;
- retry event and retry count;
- re-plan trigger;
- budget warning/exhaustion;
- terminal result.

Do not log the Groq API key. Do not log hidden chain-of-thought. Store only operationally useful summaries and observable model/tool data.

7.14 R14: Offline testability

Public and hidden unit tests must be able to replace Groq with a scripted model adapter. Reliability logic must therefore live in your Python runtime, not in an assumption that “the model usually does the right thing.”

Your normal unit tests should not consume Groq quota.

8 Framework and Library Policy

8.1 Strictly prohibited

You may not use an agent framework or a library that implements the agent runtime, graph, orchestration, planning, tool routing, multi-agent coordination, or equivalent logic for you. This includes, but is not limited to:

- LangChain and LangGraph;
- CrewAI;
- AutoGen;
- LlamaIndex agent workflows;
- PydanticAI;
- smolagents;
- Agno;
- Semantic Kernel agent orchestration;
- OpenAI Agents SDK;
- Google Agent Development Kit (ADK);
- Strands Agents;
- BeeAI or equivalent agent-runtime libraries.

The policy is based on capability, not package name. If a library is effectively implementing the controller/planner/tool router on your behalf, it is not allowed.

8.2 Allowed

General-purpose software engineering libraries are allowed. Examples include:

- Python standard library, `dataclasses`, `typing`, `sqlite3`, `hashlib`, `logging`;
- Groq SDK;
- `jsonschema`;
- `tenacity` or an equivalent general retry helper;
- `pytest`;
- `rich`;
- `requests` / `httpx` for development utilities, although the agent itself may not call external operational APIs;
- Pydantic for ordinary data validation (**not** PydanticAI);
- your own Python classes and functions.

8.3 Groq features that are out of scope

Do not use Groq built-in browser search, built-in code execution, remote MCP, or server-side agent systems. The Groq service is the LLM provider; the operational tool loop must remain visible in your own Python code.

9 AI Coding Assistants and Academic Integrity

ChatGPT, Claude, Copilot, Gemini, and similar coding assistants are **permitted**. This is intentional. The goal is not to create an assignment that depends on pretending these tools do not exist.

However, AI assistance does not change authorship responsibility. You must submit `AI_USAGE.md` using the supplied template and be able to explain and modify the submitted system during the individual TA viva.

9.1 Why copy-paste generation is insufficient

Your code will be tested against:

- unseen deterministic incident seeds;
- state changes between observation and action;
- misleading incident descriptions;
- transient tool failures;
- synthetic Groq rate-limit failures;
- malformed model tool requests;
- approval denial;
- impossible-to-fix incidents;
- reduced call budgets;
- repeated-action loops;
- false-positive action success where recovery criteria remain unmet.

A generated implementation that only passes a visible happy path will fail hidden behavioral checks and will be difficult to defend in viva.

10 Implementation Tasks

The TODO markers in the starter repository are minimum signposts, not a complete checklist. You are expected to improve the design beyond deleting TODO comments.

10.1 Task A – Re-planning policy

Implement `ReplanPolicy.should_replan()` and integrate it into the controller. Different failure classes should produce different responses. The controller should know whether it is retrying an operation, re-observing state, revising the plan, or terminating.

10.2 Task B – Loop guard

Implement stable action fingerprints and use them to stop repeated unsafe/unproductive actions. Reset or reinterpret repetition appropriately when new evidence materially changes the world.

10.3 Task C – Model retry policy

Implement bounded retry for transient model failures. A reasonable sequence is exponential or capped backoff. Unit tests must inject the sleeper so tests remain fast.

10.4 Task D – Approval integration

Read the risk policy dynamically. Before a high/critical action, request approval through the supplied gateway. Record the request and outcome. On denial, return an observation to the agent and re-plan or escalate.

10.5 Task E – Plan revision

Implement `Planner.revise()` or an equivalent explicit plan-revision mechanism. A revision must increment a counter/version and should preserve useful evidence while changing the response to new information.

10.6 Task F – Controller hardening

Improve `AgentController.run()` so that it handles all mandatory conditions in Section 7. A correct controller should not rely on one exact tool sequence.

10.7 Task G – Tests

Add at least **10 student-written tests** covering at least **five different failure classes**. At least four tests must use the scripted model adapter or another local fake so they do not call Groq.

Examples of useful tests include stale-world handling, approval denial, invalid arguments, retry exhaustion, repeated action detection, premature close rejection, re-plan after failed verification, and graceful budget exhaustion.

11 Expected Behavioral Examples

These examples describe behavior, not required exact sequences.

11.1 Example 1: deployment regression

A strong agent might inspect the incident, checkout metrics/logs, and deployment history; notice that errors began with a recent release; formulate a rollback hypothesis; request approval for rollback; execute it only if approved; verify the critical path; and close only after objective recovery.

A weak agent would see “checkout” in the ticket and immediately rollback without checking deployment evidence.

11.2 Example 2: stale world after planning

Suppose the agent observes `world_version=4` and proposes scaling checkout. Before execution, a simulated instance loss occurs and the world becomes version 5. The action returns `stale_precondition`. A strong agent re-observes relevant service state and decides whether the original scale action is still appropriate. A weak agent simply replaces 4 with 5 and retries the exact action without reconsideration.

11.3 Example 3: approval denial

Suppose evidence strongly supports database failover, but the approval gateway denies it. The agent must not bypass the human. It should revise the plan: collect additional evidence, choose a safe alternative if one exists, or escalate.

11.4 Example 4: successful action but failed recovery

A restart can return `status=ok` while the customer-facing error objective remains violated. The agent must treat `verify_recovery=false` as evidence that the incident is not resolved and must continue or re-plan.

11.5 Example 5: impossible incident

If the root cause lies outside the managed platform and all internal services are behaving consistently with an external dependency failure, repeatedly restarting internal services is harmful. Evidence-backed escalation is a correct autonomous outcome.

12 Testing and Evaluation Protocol

12.1 Public tests

The starter includes infrastructure tests that should pass before you modify the agent and student requirement tests that intentionally fail initially.

Run:

```
pytest -q tests/public -m infrastructure
pytest -q tests/public
python scripts/check_banned_imports.py
python scripts/count_todos.py
```

Do not edit tests to make them green. Instructor grading uses clean tests.

12.2 Your three required public evaluation runs

For public-a, public-b, and public-c, include in your report:

Metric	What to report
Terminal outcome	resolved / escalated / aborted / budget exhausted / failed
LLM requests	Total including planning/replanning
Tool calls	Total local environment calls
Plan revisions	Count and trigger
High-risk actions	Proposed, approved/denied, executed
Recovery proof	Verification evidence ID and close result, if resolved
Redundant calls	Any avoidable repeated observation/action
Runtime	Approximate wall-clock duration

12.3 Hidden evaluation philosophy

Hidden evaluation does not require one fixed “correct” trace. Different evidence-gathering orders can be valid. The grader primarily checks invariants: safe execution, appropriate re-planning, bounded retry, approval enforcement, correct stopping, budget discipline, and evidence-grounded terminal behavior.

13 Submission Requirements

Submit one ZIP named:

```
A1_<RollNo>_<Name>.zip
```

The ZIP must contain:

1. the complete modified source repository;
2. REPORT.md, 1,200–1,800 words, following the supplied structure;
3. AI_USAGE.md;

4. at least 10 student-written tests;
5. trace files for `public-a`, `public-b`, and `public-c`;
6. an updated README section containing exact setup and run commands;
7. a working `requirements.txt` or equivalent dependency declaration.

Do **not** submit:

- `.env`;
- API keys or tokens;
- virtual environments;
- cache folders;
- agent-framework dependencies;
- gigabytes of logs or generated artifacts.

Your submission must be runnable by the TA from a clean environment using the documented commands.

14 Marking Rubric – 100 Marks

Category	Marks	What earns strong marks
Agent runtime and explicit state	16	Clear controller boundary; correct state tracking; model proposals are separated from authoritative execution; terminal state is explicit; clean interfaces.
Planning, decomposition and dynamic re-planning	18	Useful explicit plan, meaningful subgoals/success criteria, grounded plan revisions, preserved evidence, correct response to contradictory/stale/denied outcomes.
Tool orchestration and validation	14	Strict schema/argument checking, unknown-tool rejection, correct tool-result protocol, no fabricated execution, appropriate world-version handling.
Failure recovery in a dynamic environment	14	Correct distinction among retry/re-observe/re-plan/abort; transient failures bounded; action failure and failed verification handled; no blind repetition.
Safety, approval and stopping correctness	14	Risk policy loaded dynamically; high/critical actions require genuine Python-side approval; denial respected; no false claim of success; verification + close invariant enforced.
Groq/API budget and efficiency	8	Correct accounting; robust 429/transient handling; economical observation/action choices; safe behavior under reduced budget; no quota-burning tests.
Testing, observability and engineering quality	10	Student tests cover failures; offline fake/model adapter used; trace is complete and readable; modular code; clear errors; no secret leakage.
Engineering report and AI-use declaration	6	Concise technical explanation, evaluation table, three failure traces, limitations, complete AI assistance declaration.
Total	100	

14.1 Performance is not code volume

A 400-line controller is not automatically better than a 120-line controller. Marks are awarded for correct behavior and engineering reasoning, not for producing more files or more prompts.

15 Viva / Live Verification

The TA will conduct a short individual viva. Expect approximately 5–8 minutes. The TA may ask you to:

- trace one model tool request from LLM output to validation to approval to execution to observation;
- explain why a particular failure is retried or re-planned;
- show how your loop detector works;
- explain how you prevent model claims from becoming false incident closure;
- change a risk level or call budget and show that the system adapts through configuration;
- handle one injected malformed tool call or approval denial;
- identify which part of your system would fail if a given line were removed.

The viva is not intended to reward memorization. It verifies ownership and conceptual understanding of the submitted engineering decisions.

16 Non-Compliance and Safety Conditions

The following are serious violations of the assignment contract:

- using a prohibited agent framework to implement orchestration;
- reading private/oracle simulator state at runtime;
- hard-coding seed-to-root-cause mappings;
- modifying the simulator to manufacture success;
- bypassing the approval gateway;
- disabling call limits;
- calling external systems from the incident agent;
- including an API key in the submitted repository;
- claiming a resolved incident without simulator evidence.

Technical score caps or academic-integrity procedures may apply according to course policy. The grader will use clean copies of protected files.

17 Frequently Asked Questions

Q1. Are we allowed to use ChatGPT, Claude, Copilot, Gemini, etc.?

Yes. Declare their use in `AI_USAGE.md`. You are responsible for understanding and defending everything you submit.

Q2. Can an AI coding tool generate most of my code?

There is no percentage-based ban. However, hidden tests and the individual viva measure whether the resulting system is actually correct and whether you understand it. Generated code that you cannot debug is a liability.

Q3. Can I use LangChain just for tool calling?

No. LangChain is an agent/application framework and is prohibited for this assignment.

Q4. What about Pydantic?

Ordinary Pydantic for data validation is allowed. PydanticAI is prohibited.

Q5. What about Tenacity?

Allowed. It is a general retry utility. You must still design the retry policy correctly.

Q6. Can I use SQLite?

Yes, although persistent long-term memory is not required. Do not add complexity merely to impress the grader.

Q7. Can I use another Groq model?

Yes for development if it supports the required local tool calling. The final repository must work with the instructor-specified default model unless an announcement changes it.

Q8. Do I need to use Groq structured output for the plan?

No single implementation is mandatory. You must, however, create and validate an explicit plan representation. The supplied starter shows one structured-output approach.

Q9. Can I use Groq browser search or code execution?

No. All incident evidence and actions come from the local simulator.

Q10. Can I use MCP?

No. MCP is a later course topic and is outside this assignment.

Q11. Can I use RAG or a vector database?

No need. The local runbook tool is provided. RAG is deliberately outside the scope of Assignment 1.

Q12. Do I need long-term memory?

No. You need explicit in-run state. Persistent memory is a later topic.

Q13. Do I need multiple agents?

No. This is a single-agent controller assignment. Multi-agent coordination is later in the course.

Q14. Can I build a GUI?

You may, but it carries no direct marks and must not replace required engineering work. A CLI is sufficient.

Q15. Can I use Docker?

Optional and unnecessary. The TA must still be able to run the repository using the documented normal Python setup.

Q16. Can I change the tool schemas?

Minor improvements are allowed if you preserve the stable contract and do not make hidden grading impossible. Do not remove required tools or weaken validation.

Q17. Can I add my own local helper tools?

Only if they do not reveal private simulator state, bypass risk controls, or turn the task into a hard-coded oracle. Added tools must be justified in the report.

Q18. Can I directly call `SimulationEnvironment.execute()` from my agent?

Use the supplied `ToolRegistry`. It is the validation and public execution boundary.

Q19. What exactly is `world_version`?

It is a version number for meaningful environment state. It lets the simulator reject an action based on stale observations. Think of it as an optimistic concurrency check.

Q20. If an action gets `stale_precondition`, can I simply replace the old version with the newest one and retry?

No. You must re-observe relevant state and reconsider whether the action remains justified.

Q21. What if a tool returns a transient timeout?

Use bounded retry or gather equivalent evidence another way. Do not loop indefinitely.

Q22. What if Groq returns HTTP 429?

Treat it as a retryable provider condition. Apply bounded backoff and correct request accounting. If the operation cannot succeed within the retry/budget policy, terminate safely.

Q23. Does a retry consume the assignment LLM budget?

Yes. It is still an API request.

Q24. What if the model outputs an invalid tool name?

Reject it before execution, give the model a corrective observation, and continue within budget.

Q25. What if the model stops and writes a final answer before the incident is closed?

The controller must not interpret that text as operational success. Continue/recover if safe and budget permits, or terminate as failed/escalated according to your policy.

Q26. Do I have to find a root cause before acting?

You need sufficient evidence to justify the selected action. In a real incident, mitigation can sometimes precede perfect root-cause certainty. Your report should explain the trade-off.

Q27. How much evidence is “enough”?

There is no fixed count. The evidence must be relevant to the risk of the action. A high-risk rollback based only on the incident title is weak; a correlated deployment, logs, and timing is much stronger.

Q28. Can I always ask for approval before every action?

You could, but unnecessary approvals reduce autonomy and may waste time. The policy file defines the minimum required approval boundary. Reasonable extra safeguards may be justified.

Q29. What happens after approval is denied?

The action must not run. The denial becomes new information, and your plan should change or the incident should be escalated.

Q30. What if the correct action is impossible because the incident is external?

Escalation with evidence is a correct outcome. Do not force a “resolved” status.

Q31. Why do I need `verify_recovery` after an action already returned `ok`?

Because `ok` means the requested operation executed, not that the customer problem disappeared.

Q32. Can I call `close_incident` immediately after a successful action?

No. You need a current successful `verify_recovery` result and must cite its evidence ID.

Q33. Must my tool sequence match the instructor’s?

No. Hidden grading checks safety and behavioral invariants rather than one exact sequence.

Q34. How many prompts should I use?

As many as your architecture reasonably needs within the 14-request budget. More prompts are not automatically better.

Q35. Can I place all logic in the system prompt?

No. Safety, budgets, validation, approval, retries, and stopping must be enforced in Python. Prompt instructions alone are not sufficient controls.

Q36. Can I increase the call limits?

Not for evaluated runs. Hidden grading uses the stated configuration or stricter variants.

Q37. Are public tests the full grading suite?

No. They are basic contracts. Hidden tests are deliberately adversarial.

Q38. Can I modify public tests?

You can locally experiment, but it has no effect on grading because the TA uses clean tests.

Q39. Why do tests use a scripted model?

To isolate your controller from stochastic model behavior, avoid quota consumption, and test cases where the model is deliberately wrong.

Q40. What should I do if my model makes a bad decision in one run?

Your controller should be built with the expectation that this happens. Save the trace, determine whether your Python safety/recovery mechanisms behaved correctly, and improve the architecture rather than only changing wording in the prompt.

Q41. What if Groq is temporarily unavailable near the deadline?

Your unit tests, traces from earlier runs, and framework-free code should still be inspectable. Do not wait until the final hour to perform all live runs. The final code must retain Groq support.

Q42. Can I submit my API key so the TA can run it?

No. Never submit credentials. The TA will use their own environment variable/key.

Q43. Is an elaborate prompt worth many marks?

No. Prompts matter, but this assignment primarily grades the controller, evidence flow, failure handling, safety, and evaluation.

Q44. Can I hard-code “if memory > 90 then restart” and avoid the LLM?

A small deterministic guard is allowed where it protects safety, but replacing the agent with a hard-coded incident decision tree defeats the assignment. The LLM must participate meaningfully in hypothesis/planning/action selection, while Python constrains execution.

Q45. Can my Python controller override an unsafe LLM action?

Yes. In fact, it must be able to reject unsafe, invalid, stale, unapproved, repetitive, or over-budget actions.

18 Recommended One-Week Work Plan

This schedule is advice, not an additional marking criterion.

Day	Recommended focus
1	Run infrastructure tests, understand simulator/tool contracts, map the baseline controller, make one live Groq call.
2	Implement retry policy, validation flow, risk/approval integration, and loop guard with unit tests.
3	Implement explicit plan revision and stale-world recovery; add scripted-model tests.
4	Implement terminal verification/closure logic and budget-aware fallback/escalation.
5	Run public-a/b/c repeatedly, inspect traces, remove unnecessary calls, strengthen failure handling.
6	Complete student tests, report evaluation table, three failure-trace analyses, and AI-use declaration.
7	Clean-room installation test, final live runs, secret scan, banned-framework scan, and viva preparation.

19 Final Pre-Submission Checklist

Before uploading, confirm all of the following:

- ☐ `pytest -q tests/public -m infrastructure` passes.
- ☐ Your implemented public student tests pass.
- ☐ `python scripts/check_banned_imports.py` passes.
- ☐ No API key exists in source, traces, screenshots, Git history submitted, or `.env`.
- ☐ High/critical actions really call the approval gateway.
- ☐ Stale actions cause re-observation/reconsideration.
- ☐ Groq 429/transient errors cannot create an infinite retry loop.
- ☐ Repeated identical actions are detected.
- ☐ Resolved outcomes require verification and successful incident closure.
- ☐ Impossible cases can end in safe escalation.
- ☐ Public-a/public-b/public-c traces are included.
- ☐ Report and AI assistance declaration are complete.
- ☐ You can explain every major controller transition in viva.

20 References and Current Context

These references provide context for why bounded agentic incident operations are a current engineering problem. They are not required reading for implementation.

1. Google Cloud, “AI in SRE: Where and how Google is deploying agentic AI to improve operations,” 28 May 2026. <https://cloud.google.com/blog/products/devops-sre/how-google-sre-is-using-agentic-ai-to-improve-operations/>
2. Microsoft Azure, “Azure SRE Agent.” <https://azure.microsoft.com/en-us/products/sre-agent>
3. Groq Documentation, “Tool Use Overview.” <https://console.groq.com/docs/tool-use/overview>
4. Groq Documentation, “Rate Limits.” <https://console.groq.com/docs/rate-limits>

The objective is not to make the LLM look intelligent. The objective is to engineer a system that remains useful when the LLM and the environment are imperfect.